Compiling for Distributed Memory: A Historical Survey and Taxonomy from Global Arrays to Mesh-Sharded AI Systems

Sameeran Joshi

***Abstract*—Compilers that target machines with physically distributed memories must decide where data lives, where computation runs, and what communication connects the two. These decisions were first posed as compiler problems in the late 1980s for hypercubes and multicomputers, were standardised in High Performance Fortran in the 1990s, lost ground to hand-written message passing in the 2000s, and have returned in the 2020s as SPMD partitioners and tensor-sharding compilers for accelerator meshes and wafer-scale machines. This article surveys that history. We give an operational definition of a distributed-memory compiler, a reproducible literature-discovery methodology with an openly maintained catalog, and a taxonomy with independent axes for programming abstraction, data decomposition, computation mapping, communication synthesis, compiler/runtime boundary, compiler representation, target, and optimization objective. Using the taxonomy, we trace how mechanisms such as alignment and distribution directives, the owner-computes rule, communication-set generation, redistribution, and inspector–executor scheduling have persisted, disappeared, or reappeared under new names such as device meshes, sharding annotations, partition propagation, collective insertion, and resharding. We close with open problems for heterogeneous hierarchies, dynamic and irregular partitioning, memory-capacity- and topology-aware cost models, and fine-grained spatial architectures. [TODO: Add corpus size and number of systems once the catalog is frozen.]**

***Index Terms*—Distributed-memory compilers, data distribution, owner-computes, communication generation, High Performance Fortran, inspector–executor, polyhedral compilation, SPMD partitioning, tensor sharding, wafer-scale compilation.**

I. Introduction

A machine whose memory is distributed forces a compiler to answer three questions that a shared-memory compiler can defer to hardware: where each datum is stored, which place executes each operation, and what messages must flow so that operands are present when they are needed. Compilers that address these questions for hypercube and multicomputer targets appear in the 1988 literature [1]–[3]. Within five years the answers had been standardised in the directive language High Performance Fortran (HPF) [4], and the message-passing interface MPI had been standardised as the portable alternative in which the programmer answers all three questions by hand [5]. We observe that the same three questions are now being answered again, by SPMD partitioners such as GSPMD [6] and PartIR [7], by distributed tensor-algebra compilers such as DISTAL [8], and by compilers for wafer-scale and spatial accelerators [9], often with terminology that does not acknowledge the earlier work.

Manuscript draft generated from the open catalog at https://github.com/Sameeranjoshi/dist_compilers. This is a working draft; see the repository ROADMAP for status.

A. Why the problem keeps reappearing

Every time a new class of machine with distinct memories becomes important—hypercubes, clusters, multi-GPU nodes, accelerator pods, wafers—the trade-off between programmer control and compiler automation is re-opened, and the compiler community rebuilds a stack of ownership representation, computation mapping, and communication synthesis for that class. The repository that accompanies this survey exists to make that rebuilding less amnesiac: it records, with evidence, which mechanism solved which problem, under what assumptions, and with what limitation.

B. Scope

We define a *distributed-memory compiler* as a compiler, compiler framework, programming system, or compiler-integrated runtime that transforms a program for execution across multiple computational places whose memories are physically or logically distinct at the relevant compilation level (Section II gives the inclusion criteria). Multiple GPUs are a distributed-memory target when the compiler must partition data or computation and generate communication between device memories; wafer-scale and spatial architectures count when individual processing elements have distinct local memories and data movement must be compiled explicitly. Distributed databases, distributed operating systems, cloud schedulers, hardware-only papers, and shared-memory-only compilers are out of scope.

C. Central question and research questions

The central question is: *how have compilers for machines with physically distributed memories represented, decomposed, mapped, communicated, optimized, and executed data and computation?* We refine it into ten research questions (RQ1–RQ10) on programming abstraction, data decomposition and ownership, computation decomposition, communication generation, the compiler/runtime boundary, compiler representation, optimization objectives, evaluation, historical continuity, and open problems. Section V maps each to an axis of the taxonomy.

D. Contributions

1) An operational definition and inclusion criteria for distributed-memory compilers that span 1950–2026 while keeping pre-1980 work explicitly labelled as *foundations* rather than as distributed-memory compilation (Section III).
2) A reproducible survey methodology with an open, schema-validated catalog in which every entry carries evidence pointers and a screening status (Section II).
3) A common architecture for distributed-memory compilers (Section IV) and a multi-axis taxonomy (Section V), both derived from the catalog.
4) A historical narrative organised by era (Section VI) and by mechanism (Sections VII–XI), each ending with the limitation that motivated the next step.
5) A cross-era synthesis identifying ideas that persisted, failed, or were rediscovered under new names (Section XII), and a list of open problems (Section XIII).

[TODO: We do not claim that this is the first comprehensive survey of the area; an explicit search for prior surveys is logged in the repository and the claim will be revisited once it is complete. Known prior overviews include [10]–[12].]

II. Survey Methodology

A. Scope and inclusion criteria

A system is included when it performs or represents at least one of: data distribution or ownership; computation partitioning or placement; communication generation; collective insertion; halo or ghost-region construction; global-to-local address conversion; import/export schedule generation; redistribution or resharding; hierarchical parallelism across nodes or devices; distributed execution-plan generation; or distributed-memory performance modelling used by a compiler. We classify included work as *central* (automatic distributed-memory parallelization; global-array and data-parallel compilers; Fortran D, HPF and relatives; automatic message-passing generation; distributed dense and sparse tensor compilers; SPMD partitioners; distributed GPU/accelerator compilers; spatial and wafer-scale compilers) or *bridge* (languages whose compilers implement distributed memory; compiler-integrated runtimes such as inspector–executor systems; PGAS compilers; task/region systems where placement is central; distributed-training systems with substantial plan generation; performance models that drive compiler decisions). Excluded are distributed databases and operating systems, generic job scheduling, routing protocols, hardware-only papers, application scaling studies, shared-memory-only compilers, single-GPU kernel compilers, and pure runtimes without a compiler component. A small number of architecture papers are retained in the catalog solely to date target classes; they are not classified on the compiler axes.

B. Sources and queries

Bibliographic metadata were taken, in order of preference, from the DOI record (via Crossref), arXiv, official author or institution copies, official project documentation, and trusted bibliographic indexes. No paywall was bypassed; where full text was not available the catalog marks the entry `paywalled` or `abstract-only` and no mechanism is inferred from the title. The initial query set (25 phrases such as “data distribution compiler”, “communication set generation”, “inspector executor distributed memory”, “SPMD partitioner compiler”, “wafer scale compiler”) is listed in the repository together with a discovery log recording each query, date, hits, and screening decision. [TODO: Report per-database hit counts once the full query sweep over ACM DL and IEEE Xplore is logged.]

C. Snowballing

From every anchor paper we snowball backward (references), forward (citing works, prioritising those that extend, critique, evaluate, or revive the anchor), laterally (same system, project, or authors), across surveys (works cited by several independent surveys are prioritised), and by *gap*: for every empty cell of the taxonomy coverage matrix we search for that combination specifically.

D. Screening and evidence levels

Each catalog entry carries one of five statuses: *discovered* (metadata only), *screened* (abstract and introduction checked), *core* (clearly relevant), *read* (enough read to summarise accurately), and *verified* (metadata, categorisation, and summary reviewed against the primary source). Only *read* and *verified* entries support mechanism-level claims in this article. A paper is called a survey only if it is a survey or an invited overview; for example, Lee’s “Techniques for compiling programs on distributed memory multicomputers” [13] is catalogued as a technique paper.

E. Reading strategy and paper cards

For most papers we read the abstract, introduction, system overview, related work, and conclusion; we read the full paper when it introduced a major system, defines a taxonomy category, supports a key historical claim, has experiments that we compare, or supplies a mechanism central to the synthesis. Every core paper receives a paper card in the repository listing problem, input abstraction, target, decomposition, mapping, communication, compiler/runtime split, evaluation, limitation, lineage, and the exact sections used.

F. Evidence labelling

Statements in this article are of three kinds: source-supported facts, which carry a citation; survey taxonomy introduced by us, flagged by “we categorize” or “we group”; and inferences, flagged by “appears”, “we observe”, or “we hypothesise” and listed with their current evidence in the repository’s historical-claims file. Conflicts between sources are recorded rather than resolved silently.

G. Limitations

Full text of many 1988–2000 papers is paywalled; their summaries rely on open author copies where available. Citation counts are not used as a relevance criterion. Dates of era boundaries are provisional and are justified by the dated events listed in the catalog.

III. Foundations (c. 1950–1979)

This section separates what the distributed-memory compilers of the late 1980s inherited from what they invented. Nothing here is a distributed-memory compiler. The machines of the period had a single address space or lock-step SIMD control, and placing data in memories was not a compiler concern. Where a foundations paper has no card in the repository yet, we state only its bibliographic record and mark the content claim as pending.

A. Automatic programming

The FORTRAN automatic coding system established that a translator could generate object code for numerical loops from a higher-level notation [14]. We take this as the first instance of the bargain that every later system renews: the programmer writes a higher-level program, and the compiler makes the low-level decisions. [TODO: Add a card for [14] and quote its code-quality claims only after reading.]

B. Dependence analysis and loop parallelism

Lamport's paper on the parallel execution of DO loops, published in 1974, is the standard reference for reordering nested loop iterations so that independent iterations run together [15]. Kuck and colleagues presented dependence graphs as the representation on which compiler optimisation and parallelisation are built [16], and Allen and Kennedy translated FORTRAN loops into vector form using the same analysis [17]. Distributed-memory compilers reused this dependence representation without change for the iteration and access analysis; what they added was an ownership function. Section VII shows this for the Fortran D and PARADIGM lineage. [TODO: Read [16] and [17] and replace this paragraph with card-backed statements.]

C. Machine abstractions

The ILLIAC IV array processor [18] and the CRAY-1 vector computer [19] are the two regular-parallel machine classes that the early loop compilers targeted. Dennis and Misunas proposed a dataflow processor architecture in 1975 [20], and a 1977 MIT workshop report on dataflow computers and program organisation is in the catalog from a supplied copy [21]. Kung's article on systolic architectures [22] and the Cosmic Cube multicomputer [23] establish the two architectural ancestors of the spatial and distributed targets discussed in Sections XI and VI. [TODO: Read [22]: the supplied copy has no text layer and must be read visually before any content claim is made.]

D. Data-parallel programming

Hillis and Steele described data-parallel algorithms on the Connection Machine, in which one operation applies to all elements of a data set at once [24]. This is the programming model that Fortran 90 array statements and, through them, HPF inherited. [TODO: Verify the lineage claim against the HPF specification [25] once that record is read.]

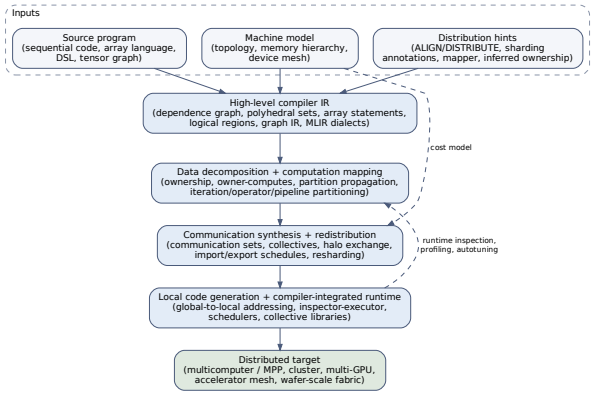


Fig. 1. The stages that recur across the catalog's distributed-memory compilers. Most 1990s systems are linear through the communication stage. The dashed edges are where systems differ on the compiler/runtime boundary (axis F): runtime inspection, profiling and cost models feed back into decomposition. The figure is original to this survey and is abstracted from the cited systems.

E. Synthesis

The foundations supplied three things to the next era: a trusted pipeline for transforming loops, a legality test for reordering them, and machine models with regular parallel structure. What they did not supply was any representation of where data resides. That omission became the central compiler problem of the late 1980s, once machines with private memories appeared [23].

IV. A Common Compiler Architecture

We abstract the systems in the catalog into the pipeline of Fig. 1. The inputs are a program (sequential code, an array language, a domain-specific language, or an operator graph), a machine model (processor arrangement, device mesh, memory hierarchy), and distribution information. That information is either supplied by the programmer, as ALIGN and DISTRIBUTE directives [4], sharding annotations [6], or a task mapper [26], or it is inferred.

a) Intermediate representation: The representation determines which decisions can be made statically. Vienna Fortran's distributions are declarations attached to arrays [27], and Kali's distributions are declared with processor arrays and on-clauses [28]. Koelbel's closed forms are derived from the subscripts of forall loops [29], and Chatterjee et al. give a finite-state representation of local access sequences for cyclic(k) distributions [30]. The polyhedral framework represents iteration and access spaces as integer sets and was used for MPI generation by Bondhugula [31]. Mesh-TensorFlow names tensor dimensions and maps them onto a processor mesh [32], and MLIR's shard dialect represents grids and per-axis shardings as IR attributes [33].

b) Data decomposition and computation mapping: Kali's doall loops execute an iteration on the processor named by an on-clause, and the report describes owner-based placement [28]. Chen et al. compile Crystal programs, whose

data fields carry communication metrics, by reshaping them with morphisms [3]. In GSPMD, sharding annotations are completed by propagation through the graph with priorities, and each operator is then partitioned [6]. Each approach derives a computation mapping from a data mapping, though the direction of derivation and the place where the decision is made differ.

c) Communication synthesis and redistribution: Communication can be derived in closed form from the distribution functions [29], [30], recognised as collectives, or determined at run time from index arrays [34]. Redistribution between distributions is handled by pairwise communication plans in Ramaswamy et al. [35], and by phase-level planning in Palermo et al. [36]. In MLIR's shard dialect, collectives such as all-gather, all-reduce and all-to-all are operations with the participants inferred from the sharding [33].

d) Local code generation and runtime: The PARTI/CHAOS line supplied schedules for off-processor accesses [37], [38]. Legion executes a task graph over logical regions and maps them to memories through a mapper [26]. Rogers and Pingali emitted per-process C code with run-time resolution as a fallback [39]. The amount of work delegated to the runtime is axis F of the taxonomy.

e) Synthesis: The pipeline has been stable since the early 1990s. What has changed is the input abstraction (axis B), the representation on which decomposition is decided (axis K), and which stages are static (axis F). These three differences are the axes along which the eras of Section VI separate.

V. Taxonomy

We classify each paper or system along eleven independent axes (Fig. 2): (A) target and memory model; (B) programming abstraction (RQ1); (C) data decomposition (RQ2); (D) computation mapping (RQ3); (E) communication synthesis (RQ4); (F) compilation time, i.e. the compiler/runtime boundary (RQ5); (G) compiler architecture; (H) optimization objective (RQ7); (I) maturity and artifact status; (J) paper type; and (K) compiler representation (RQ6). The value lists are maintained in the repository and enforced by a validator.

A. Why chronology alone is insufficient

HPF (1993) and GSPMD (2021) take the same values on axis C (block distribution over a logical processor grid) and on axis E (communication inserted by propagating distributions through the program), yet they are three eras apart; inspector–executor runtimes (1990) and distributed sparse-tensor compilers (2022–2025) share the *runtime inspector* value on axis F. A chronological list hides these recurrences, which are among the survey's main findings (Section XII). Chronology is therefore one dimension (Section VI), used to explain *why* a value was chosen at a given time, while the axes explain *what* was chosen.

B. Provenance of category names

Where a standard term exists we use it: *owner-computes* from the Kali/Fortran D literature, *inspector–executor* from PARTI, *BLOCK*, *CYCLIC*, and *block-cyclic* from HPF, *partition propagation* from GSPMD, *inter-* and *intra-operator parallelism* from Alpa [40]. The grouping of these into axes, and the decision to place ML sharding compilers and wafer-scale compilers on the same axes as 1990s HPF compilers, is ours.

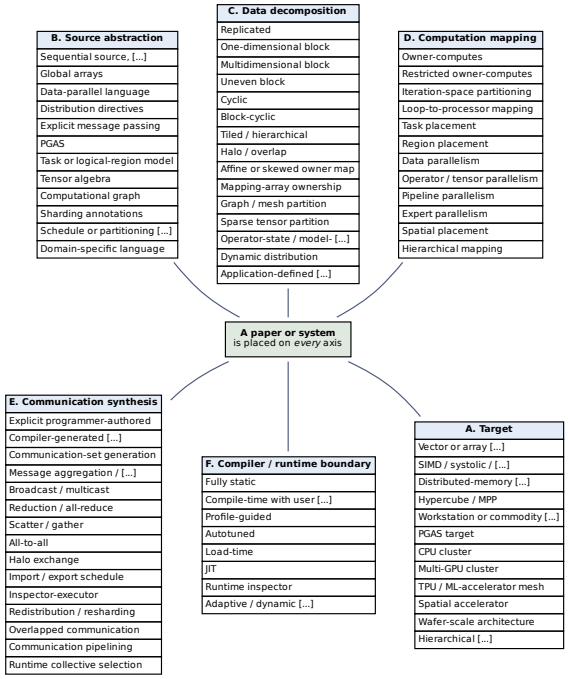


Fig. 2. The taxonomy is a set of orthogonal axes, not a tree: each paper or system takes one or more values on every axis. Only the six axes used most in the text are shown; the catalog carries eleven. Generated from the machine-readable taxonomy so that the figure cannot drift from the catalog.

C. Tagging discipline

A tag is attached to a catalog entry only when an evidence pointer names the section of the primary source supporting it. Entries whose full text has not been consulted are tagged only on the axes their abstract clearly supports. Lineage fields record only relationships that a paper states itself or that a cited survey states; inferred lineages are kept separately. Table I summarises the eras, Table II the core systems, and Tables III–V the decomposition support, compiler architecture, and evaluation methodology of each system; all are generated from the catalog.

VI. Historical Evolution

Fig. 3 and Table I organise the catalog into seven provisional eras. Each subsection ends with what changed, what remained, and the limitation that motivated the next era.

A. Era 1: emergence (c. 1980–1987)

Message-passing multicomputers made private memories and explicit communication the programmer's responsibility;

TABLE I
ERA SUMMARY. DATES ARE PROVISIONAL (SEE SEC. II); REPRESENTATIVE SYSTEMS ARE THE CATALOG ENTRIES WITH THE HIGHEST READING PRIORITY IN EACH ERA.

Era	Machine driver	Programming abstraction	Decomposition representation	Generated communication	Representative systems	Limitation
0: Foundations (1950–1979)	Stored-program machines, FORTRAN, pipelined vector processors, lock-step array processors, dataflow and systolic proposals	Sequential FORTRAN; vector statements; early parallel-language proposals	None for distribution; dependence graphs and loop-level analysis for vectorization	None (shared or centralised memory)	–	No notion of data placement as a compiler problem; machines had a single address space or SIMD broadcast control
1: Emergence of distributed-memory compilation (1980–1987)	Multicomputers and hypercubes (Cosmic Cube, iPSC, nCUBE), systolic arrays, Connection Machine	Explicit message passing; early global-array proposals; SIMD data-parallel languages	Programmer-managed; first compiler proposals for placement	Hand-written sends/receives	–	Programming by hand was error-prone; portability across machines was poor
2: Global arrays and data decomposition (1988–1992)	Commercial hypercubes and MPPs (iPSC/2, iPSC/860, nCUBE-2, CM-2)	Sequential or global-array programs with distribution annotations (SUPERB, Kali, Vienna Fortran, Fortran D)	BLOCK/CYCLIC distributions, alignment to templates/decompositions, overlap regions	Compiler-generated point-to-point messages; message vectorization; first inspector-executor runtimes	[1], [41]–[43]	Regular, affine programs only; automatic distribution selection intractable in general; communication costs poorly modelled
3: HPF, MPI, and compiler maturity (1993–2000)	MPPs (CM-5, Paragon, SP2, T3D/T3E) and early clusters	HPF directives; MPI library calls; irregular extensions	HPF ALIGN/DISTRIBUTE with BLOCK, CYCLIC(k), templates and processor arrangements; INDIRECT/mapping arrays in HPF-2 and research systems	Communication sets, local address generation, redistribution, collectives; PARTI/CHAOS schedules for irregular code	[4], [10], [11], [44]	HPF compilers were slow to mature and performance was unpredictable; explicit MPI became the dominant practice
4: MPI dominance and alternative programming models (2000–2010)	Commodity clusters, early multicore nodes, Cell/BE and explicitly managed memories	MPI(+OpenMP); PGAS (UPC, Co-array Fortran, Titanium); HPCS languages (Chapel, X10); hierarchical memory models (Sequoia)	Language-level distributions and locales/places; hierarchical task trees	One-sided communication, compiler-scheduled bulk transfers for memory hierarchies	[45]–[48]	Adoption of new languages remained limited; automatic distribution rarely used in production
5: Compiler/runtime co-design and accelerators (2010–2019)	GPU clusters, heterogeneous nodes, early TPU pods	Affine loop nests; task/region programs (Legion, Regent); tensor DSLs; ML graph frameworks	Polyhedral partitions; logical regions and partitions; named-dimension layouts (Mesh-TensorFlow)	Polyhedral communication-set generation with MPI; dynamic region coherence; collectives in ML frameworks	[26], [31], [49], [50]	Each system covered one domain; no shared ownership IR; ML partitioning still largely manual
6: Mesh sharding, ML systems, and spatial accelerators (2020–2026)	TPU/GPU pods, AI accelerators, wafer-scale engines, spatial dataflow chips	Sharding annotations on device meshes; tensor algebra with distribution formats; MLIR dialects; stencil DSLs	Device meshes with per-dimension sharding; distributed tensor formats; MLIR shard/mesh attributes	Collective insertion by propagation; resharding; automated search over parallelism plans	[6]–[8], [40]	Dynamic/irregular workloads, heterogeneous hierarchies, and memory-capacity constraints remain weakly handled; fragmentation across frameworks

the Cosmic Cube is the reference machine of this class [23]. [TODO: Add the early source-to-source and placement systems of this period once their records exist.] *Synthesis.* What changed: programmers faced private memories. What remained: sequential loop analysis. Limitation: all communication was hand-written, which motivated the compiler efforts of the next era.

B. Era 2: global arrays and data decomposition (c. 1988–1992)

Zima, Bast and Gerndt's SUPERB system is described as a tool for semi-automatic MIMD/SIMD parallelisation [2]. [TODO: Read [51] and state what it contributes to the handling of non-local copies in local computation.] Kali compiled global-name-space parallel loops over explicitly distributed arrays [28], [41]. Vienna Fortran extended Fortran with distribution, alignment and dynamic-distribution declarations [27], [54]. Fortran D introduced decompositions and alignment with optimisations such as message vectorisation, and the 1992 evaluation measured the effect of each optimisation on an iPSC/860 [53], [71]. Gupta and Banerjee chose one distribution per array by combining constraints weighted by estimated execution-time penalties [55]. Koelbel derived closed forms for communication and iteration sets of forall loops over block and cyclic arrays [29]. The PARTI runtime generated schedules for irregular loops whose access patterns are known only at run time [34], [52]. *Synthesis.* What changed: data ownership became a compiler-visible object, and communication moved from the programmer into the compiler. What remained: dependence analysis from the foundations. Limitation: the systems handled regular affine code, and the choice of distribution was left to a costly or manual process, which motivated the standard language of the next era.

C. Era 3: HPF, MPI, and compiler maturity (c. 1993–2000)

HPF 1.0 standardised ALIGN, DISTRIBUTE, TEMPLATE and PROCESSORS, with FORALL and INDEPENDENT [4], [25]. Benkner and Zima's 1999 overview describes the compilation techniques of the decade: distribution and alignment

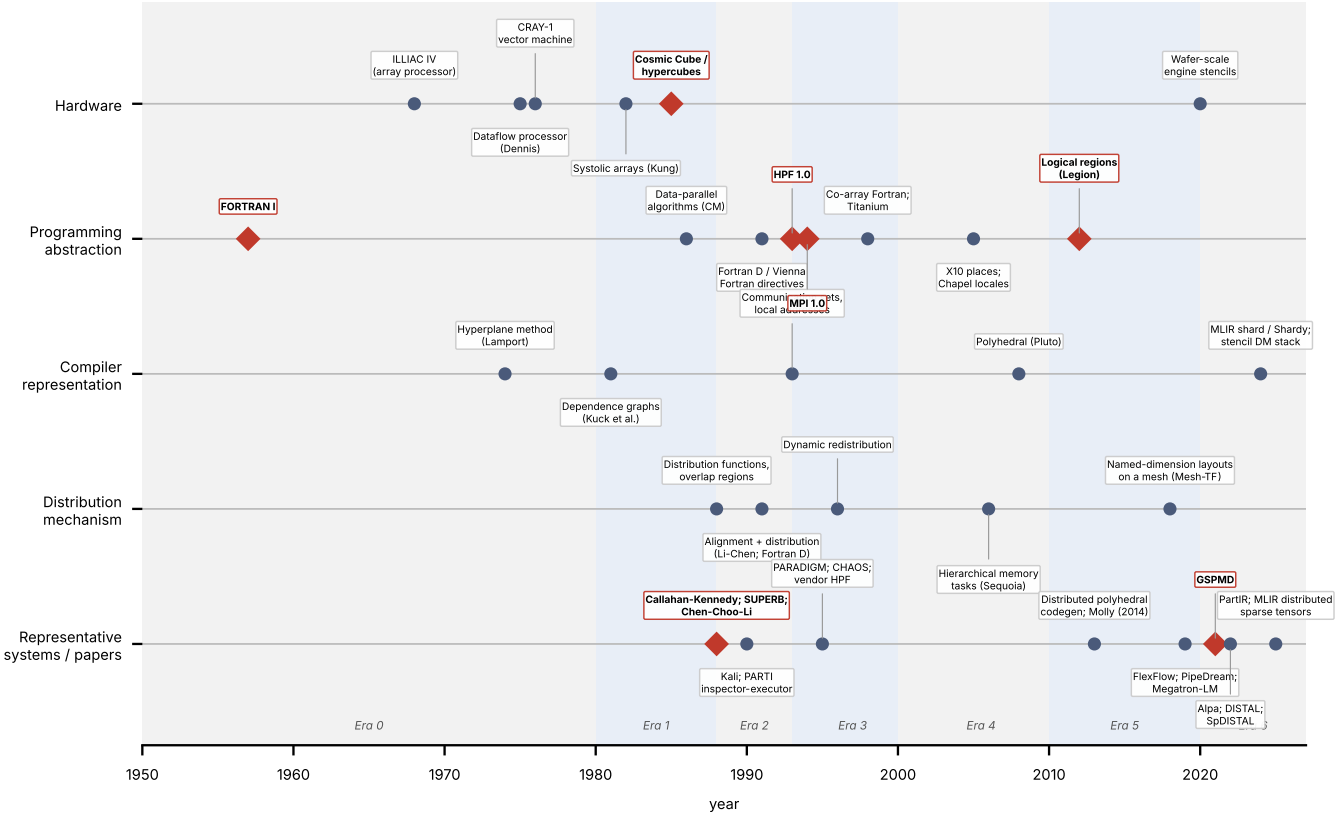


Fig. 3. Timeline 1950–2026 with lanes for hardware, programming abstraction, compiler representation, distribution mechanism, and representative systems. Diamonds mark transitions that open a new era. Each event is tied to a catalog entry; era boundaries are provisional.

processing, local index sets, owner-computes translation of array statements, and an inspector/executor scheme for irregular code [11]. Research compilers in this period include Fortran 90D/HPF [57], PARADIGM [44], [72] and Rogers and Pingali's Id-based compiler [39]; Digital's HPF 1.1 compiler was a production product [58], and IBM's pHPF was a prototype for SP machines [73]. Closed-form communication sets for block-cyclic distributions were given by Chatterjee et al. [30], and the redistribution problem was analysed by Ramaswamy et al. [35] and Palermo et al. [36]. MPI was standardised as the explicit alternative [74], with MPICH as a portable implementation [5]. Kennedy, Koelbel and Zima's retrospective gives the causal account of why HPF lost most of its user base [75], and Loveman's article lists irregular and user-defined distributions among HPF's missing capabilities [4]. *Synthesis.* What changed: a standard distribution language and mature closed-form communication generation. What remained: owner-computes and block and cyclic distributions. Limitation: irregular codes were poorly served, and compiler performance was unpredictable, so practitioners used MPI directly.

D. Era 4: MPI dominance and alternative models (c. 2000–2010)

Co-Array Fortran extends Fortran 95 with co-dimensions that name remote data by image index [45]; Titanium is a Java dialect with a partitioned global address space and domain-based arrays [76]; and the Berkeley UPC compiler translates UPC to C over GASNet [48]. Chapel presents user-definable distributions over locales as a global-view alternative to fixed directive sets [60]. Sequoia programs are hierarchies of tasks whose arguments describe data movement through a model of machine memory [61], and Baskaran et al. allocate scratchpad buffers and generate movement code for explicitly managed memories [77]. Tang and Xue generate SPMD code for rectangularly tiled, skewed iteration spaces with the computer-owns rule [47]. The Pluto polyhedral optimiser [62] provides the transformations that later distributed-memory generators build on. *Synthesis.* What changed: distribution moved from directives into language types, domain maps and memory hierarchies. What remained: the owner concept. Limitation: adoption stayed limited, and automatic distribution was rarely used in production. [TODO: Add the Berkeley UPC and HPCS evaluations, and verify the adoption claim against [75].]

E. Era 5: compiler/runtime co-design and accelerators (c. 2010–2019)

Bondhugula generated MPI communication for affine loop nests from polyhedral dependences, with flow-out and write-out sets and run-time helper functions for partners [31], [50]; Molly does the same inside LLVM and Polly with element-level home locations [49]; and Dathathri et al. target

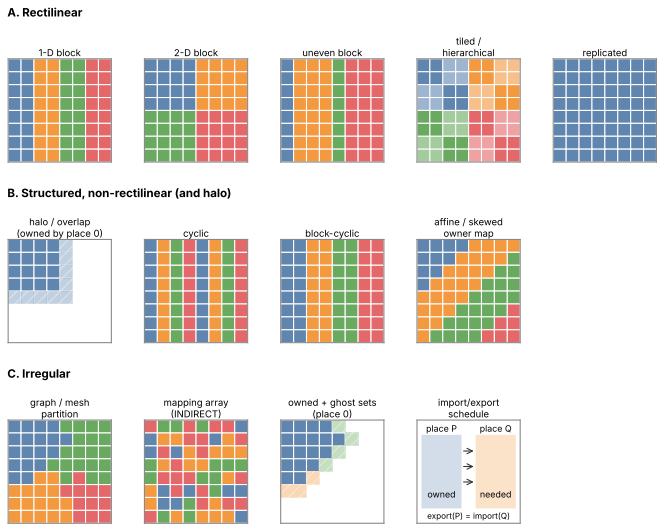


Fig. 4. Data-decomposition taxonomy (axis C) drawn as example grids coloured by owning place. Rectilinear forms (A) have closed-form owner functions. Structured non-rectilinear forms (B) need block-cyclic address arithmetic. Irregular forms (C) need a mapping array or graph partition and, usually, run-time schedules. Hatched cells are ghost copies. The examples are illustrative, not drawn from a specific system.

a dataflow runtime [78], [79]. Legion expresses locality and independence with logical regions and a mapper interface [26], and Regent compiles a sequential-looking language with privilege checking onto Legion [63]. Mesh-TensorFlow introduced named-dimension layouts on a processor mesh [32]; FlexFlow searched per-operator parallelisation configurations with a simulator [64]; GPipe and PipeDream established pipeline parallelism for training [80], [81]; and Megatron-LM implemented tensor parallelism by hand [82]. *Synthesis.* What changed: accelerators and ML workloads became the driver, and representations diversified into polyhedral, region-based and mesh forms. Limitation: each system covered one domain, and the plan for an ML model was still largely a manual choice.

F. Era 6: mesh sharding, ML systems, spatial accelerators (c. 2020–2026)

[TODO: Read [65] and state its contribution from the text; it is catalogued as the predecessor of GSPMD.] GSPMD generalised annotation completion and per-operator SPMD partitioning [6]. Alpa combines inter- and intra-operator parallelism [40], and PartIR composes partitioning strategies as tactics [7]. *Synthesis.* What changed: the programmer writes an unpartitioned model and a mesh, and the compiler propagates the layout. Limitation: dynamic, irregular and memory-capacity-limited workloads remain weakly handled; Section XIII returns to this.

VII. Data Decomposition and Computation Mapping

A. Representing ownership (RQ2)

Fig. 4 groups decompositions into rectilinear, structured non-rectilinear and irregular forms. Fortran D and HPF express ownership by aligning arrays to a template and distributing the template over a processor arrangement, using BLOCK, CYCLIC and CYCLIC(k) [4], [25]. Koelbel and Mehrotra's Kali treats distributions as declared properties of arrays over processor arrays, and adds user-defined map functions [28]. Vienna Fortran's distribution language includes user-defined distribution functions, alignment with linear expressions, INDIRECT mappings and dynamic redistribution with connect sets [27], [54]. Chatterjee et al. represent the local access sequence of a cyclic(k) distribution with affine alignment as a finite-state machine [30]. Gupta and Banerjee choose one distribution per array from weighted constraints [55], and PARADIGM extends this from block to cyclic and block-cyclic distributions by symbolic partitioning [72]. Lee's compiler generalises the distribution function and selects layouts across consecutive loops by dynamic programming [13]. Irregular ownership is expressed through a mapping array and partitioner-driven translation in CHAOS, whose partitioner can work from connectivity, coordinates or loads [83]. Chandra et al. add regular and reshaped array distributions as directives on a cache-coherent NUMA multiprocessor, where the data layout is not a message-passing layout [84]. Modern systems express ownership as per-dimension sharding over a mesh [6], [33] or as tensor distribution formats [8].

B. Assigning computation (RQ3)

Owner-computes assigns an assignment to the owner of its left-hand side. Kali expresses it through on-clauses on doall loops [28], and the Fortran D compiler applies it to partition node programs [53]. Koelbel's closed forms partition the iteration set of a forall loop by the same owners [29]. PARADIGM partitions iterations and data by symbolic analysis [72] and, in the 1995 overview, combines data and functional parallelism [44]. Legion places tasks and regions through a mapper [26], and Sequoia maps tasks to memory levels through a per-machine specification [61]. ML systems choose among data, operator, pipeline and expert parallelism; Alpa chooses among them per stage with an integer program and dynamic programming [40], and FlexFlow searches per-operator partition configurations [64].

C. Synthesis

Across these systems, the choice of computation mapping follows the choice of data mapping in most cases, and the systems differ in where the choice is made: in the programmer's directives (HPF, Kali), in a closed-form compiler analysis (Koelbel, Chatterjee), in a search over configurations (FlexFlow, Alpa), or in a mapper (Legion). [TODO: Compare the expressiveness of HPF's distribution language with GSPMD sharding and DISTAL formats using the specifications [6], [8], [25].]

VIII. Communication Generation and Redistribution

Koelbel derived send, receive and iteration sets in closed form for forall loops over block and cyclic arrays with linear

subscripts, and applied them in the Kali compiler [29]. Chatterjee et al. generate local addresses and communication sets for cyclic(k) distributions from a finite-state machine [30]. Shen et al. revisit the Amarasinghe and Lam communication-set algorithms and remove iteration variables that do not change which element or processor is involved; on one LU kernel this reduces generated communication [85], [86]. Fortran D's optimisations include message vectorisation for block-distributed arrays [53], and the 1992 evaluation classifies them by the cost they reduce: startup, copy and transit time, or lack of parallelism [71]. Tang and Xue aggregate communication per tile after skewing [47].

A. Redistribution

Ramaswamy, Simons and Banerjee represent each regular distribution as a family of line segments, so the data that must move between two processor sets is an intersection of families; they add a locality-aware packing scheme and a contention-reducing send order [35]. Palermo et al. extend PARADIGM's static partitioner with a recursive decomposition of a program into phases and a shortest-path search over phase transitions that accounts for redistribution cost [36]. Lee's compiler chooses data layouts across consecutive loops by dynamic programming [13]. Vienna Fortran's DISTRIBUTE statement redistributes arrays at run time [27].

B. Polyhedral and element-level generation

Bondhugula's generator computes flow-out and write-out sets per tile, packs them, and uses compiler-generated helper functions that evaluate the processes that must receive data at run time [31], [50]. Molly derives asynchronous chunked MPI transfers from flow dependences, using home locations for array elements [49]. Dathathri et al. report minimal communication volume for most dependence patterns in a dataflow runtime [78].

C. Collectives in SPMD partitioners and tensor compilers

GSPMD inserts collectives where propagated shardings disagree [6], and MLIR's shard dialect exposes all-gather, all-reduce, all-to-all, reduce-scatter and halo update as operations [33]. Kong et al. restrict generated code to collectives: a cost-model constraint forbids the crossed mappings that would need point-to-point traffic, and each (operator, tensor) communication vector selects a sub-communicator [87]. DISTAL expresses communication through its communicate command, and the Legion runtime realises it from partitions [8]. Vaidya et al. implement get, set and update-slice operations over MPI all-to-all and all-to-allv [88].

D. Halo exchange and overlap

Bisbas et al. lower halo exchanges, declared in the dmp dialect, to non-blocking sends and receives in the MPI dialect; each halo is a separate exchange, and the authors note that overlap and diagonal exchanges are missing [68]. On the Cerebras engine, Stawinoga et al. convert halo exchanges into chunked prefetches that overlap with computation [69]. Rocki et al. overlap receiving and multiplying in background threads, while their all-reduce remains blocking [9].

E. Synthesis

Communication generation has moved from element-level sets, derived from affine subscripts, to collectives derived from operator semantics. The first preserves generality over index expressions; the second is simpler to model because it only needs operator-level patterns. Halo exchange has been the most stable pattern: it appears in HPF overlap areas, in SUPERB-style overlap, in the dmp dialect and in the wafer-scale runtime. [TODO: Quantify the communication-volume effect of collective insertion against element-level generation using the evaluations of [6], [7], [87].]

IX. Runtime Analysis and Irregular Computation

Saltz and colleagues separated the work of an irregular loop into a preprocessor that builds communication pairs and translates global indices, and an executor that runs a table-driven schedule; the 1989 report measures executor overhead on an iPSC/2 at about 9–10% for sparse matrix-vector multiplication and higher for multi-phase triangular solves [34]. The 1990 report extends this to a hashed cache and a distributed translation table, describes a prototype compiler (ARF) that generates PARTI calls for irregularly distributed arrays, and introduces the inspector and executor vocabulary [52]. Das et al. add incremental schedules that fetch each off-processor datum once across several loops, and a mapper coupler that gives graph partitioners a runtime dependence graph [37]. Ponnusamy et al. let the user name a partitioner, and reuse inspector results when indirection arrays are unchanged, using a conservative test [83]; Hwang et al. add light-weight and two-phase schedules for adaptive irregular programs [38]. Kali's inspector results are cached and reused across repeated executions [28].

Rogers and Pingali combine run-time resolution, which is correct for any reference, with compile-time resolution where ownership is analysable [39]. Legion moves data-movement decisions to a runtime that reasons about logical regions and privileges [26]. SpDISTAL derives ownership of sparse structures as partitions, using image and preimage operations at run time [67].

A. Synthesis

The runtime boundary separates systems more sharply than any other axis. Fully static, closed-form systems [29], [31] are fast and predictable but restricted to affine code. Runtime-inspected systems [52], [83] handle indirection at the cost of an inspector that must be amortised over repeated executions. Dynamically scheduled runtimes [26], [79] move the decision further from the compiler. The records of Kali, PARTI and CHAOS show the same trade-off: schedule reuse is what makes the inspector affordable.

X. Performance Modeling and Optimization Objectives

Cost models appear in the catalog in three forms: analytic models of a compiler's choices, profiles that feed a partitioner, and search over configurations scored by a simulator or a solver.

A. Analytic models

[TODO: State what the performance-optimisation approach of [3] measures, from the paper's text; the card records the Crystal language but not yet its cost model.] Gupta and Banerjee weight distribution constraints by estimated execution-time penalties, and note that the combined problem is NP-hard and that their estimates rest on compile-time assumptions about loop bounds and branch probabilities [55]. Palermo et al. describe their static pipeline cost model as conservative [36]. Kong et al. minimise local work plus a weighted communication volume under per-node memory capacity; their cost model is volume-only, without latency or bandwidth terms, and the paper reports no numeric speedups in its text [87].

B. Profiles and simulators

PipeDream's optimiser uses a single-GPU profile of per-layer compute time, activation size and parameter size, and a dynamic program over a hierarchical bandwidth model that minimises the time of the slowest stage [81]. FlexFlow's simulator profiles each operator once and re-simulates incrementally during a Markov-chain search over per-operator configurations; the authors state that the simulator assumes operator times are predictable and independent of input data [64].

C. Search and hierarchical optimisation

Alpa solves intra-operator parallelism with an integer program over enumerated algorithms with resharding costs, and inter-operator parallelism with dynamic programming over profiled stage costs [40]. PartIR's automatic tactic uses an analytical simulator and Monte Carlo tree search, and the propagation itself uses no cost heuristics [7].

D. Objectives

Table-free summary of axis H as the catalog records it. Communication volume is the objective of the analytic and search systems above [64], [87]; load balance and pipeline stage time dominate the training planners [40], [81]; local memory fit is the constraint in [87] and the motivation for model-state sharding in [89]; and productivity is the claimed benefit of the declarative systems [7], [32]. [TODO: Tabulate the objective of each core system with evidence pointers once all cards are merged; the catalog currently carries the tags but not the comparison.]

E. Synthesis

Analytic models were abandoned for automatic distribution in the 1990s, and the modern systems that search do so over operator-level choices, where profiling is cheap. [TODO: Test this against the records of Gupta and Banerjee and Palermo et al. before stating it as a finding.]

XI. Multi-GPU, ML, Spatial, and AI-Accelerator Compilers

Mesh-TensorFlow expresses a program as named tensor dimensions mapped onto a mesh of processors, so any dimension can be split and the collectives follow from the layouts [32]. GSPMD completes sharding annotations by priority-based propagation and partitions every operator, covering data, model, spatial and pipeline parallelism through one mechanism [6]. Its authors report 50–62% TPUv3 utilisation on up to 2048 cores and models with up to one trillion parameters, without a cost model or search [6]. PartIR separates partitioning tactics from the model and composes them on an MLIR-based IR [7]. OpenXLA's Shardy combines the GSPMD and PartIR approaches [70], and the MLIR shard dialect, formerly called mesh, is described as inspired by GSPMD [33].

Automatic search is represented by FlexFlow [64] and Alpa [40]. Pipeline training is represented by PipeDream, which partitions layers into stages from a profile and schedules them with weight stashing [81], and by GPipe [80]. Megatron-LM implements tensor parallelism by hand [82], and ZeRO is catalogued for model-state sharding [89]; its card is pending. We classify the last four as bridge entries because their plan generation is manual or limited.

DISTAL and SpDISTAL bring distribution formats and scheduling commands to dense and sparse tensor algebra, compiled to Legion [8], [67]. Kong et al. derive distributed-memory mappings for tensor DAGs automatically [87]. Vaidya et al. express distributed sparse tensors as an MLIR dialect that lowers to MPI [88].

On spatial and wafer-scale targets, Rocki et al. hand-map a BiCGStab solver onto the Cerebras engine with one mesh column per core and a hardware-routed AllReduce [9]; Jacquelin et al. study scalable distributed high-order stencil computations [66] (the machine and results are pending a read of the paper); and Stawinoga et al. compile stencil DSLs to the Cerebras software language through three new dialects [69]. Bisbas et al. give stencil DSLs distributed-memory parallelism on clusters through a shared stack [68].

[TODO: Add a card for Tiramisu [90], which the catalog lists with distributed-memory support, and for the 2024 Shardy and MLIR documentation once read in full. The accelerator-vendor compilers named in the brief (Graphcore, SambaNova, Tenstorrent, Groq) are not yet catalogued.]

XII. Cross-Era Synthesis

A. Ideas that persisted

Block distributions over a logical processor arrangement, owner-based assignment of work, and the separation of a global index space from local storage appear in Fortran D [42], Vienna Fortran [54], HPF [25], and in mesh and sharding systems [6], [32]. Fig. 5 pairs the older concepts with their descendants.

B. Ideas that receded

Automatic distribution selection for general Fortran programs receded after the early 1990s. The 1992 Gupta and

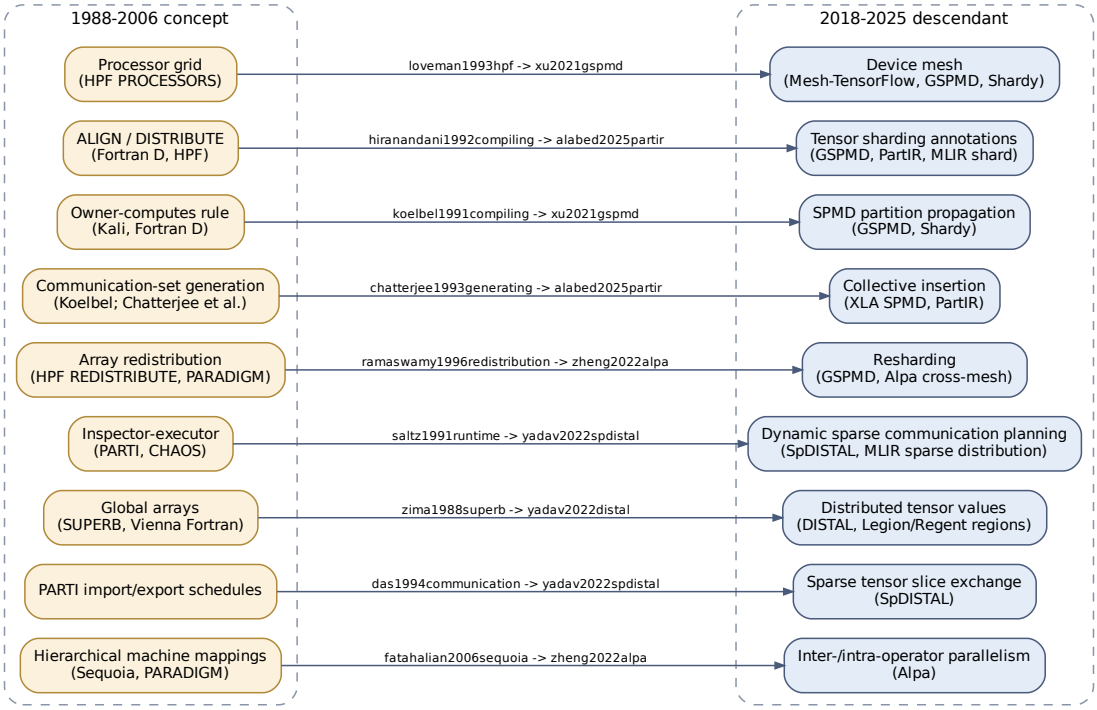


Fig. 5. Concepts of 1988–2006 and their 2018–2025 descendants. Each edge names the catalog entries that document the older and the newer form. The pairing is a survey hypothesis: it asserts functional correspondence, not that the newer work cites the older one. Evidence for each pair is tracked in the repository's historical-claims file.

Banerjee formulation was NP-hard in general and relied on compile-time estimates [55]; PARADIGM's later work still delegated communication code generation in part to the programmer [72]. Kennedy, Koelbel and Zima's retrospective attributes HPF's decline to the factors discussed in Section VI [75]. [TODO: Confirm the attribution by reading [75] in full.]

C. Concepts rediscovered under new names

Processor grid → device mesh; ALIGN/DISTRIBUTE → sharding annotation; owner-computes → partition propagation; communication sets → collective insertion; redistribution → resharding; inspector–executor → run-time sparse communication planning; global arrays → distributed tensor values; PARTI import/export schedules → sparse slice exchange; hierarchical machine mapping → inter-/intra-operator parallelism. The inspector–executor split is documented in the 1990 PARTI reports [34], [52], and its sparse descendant appears in SpDISTAL [67]. Chatterjee et al.'s local address and communication-set generation [30] and the mesh shardings of MLIR [33] both answer which elements a processor must touch under a distribution. [TODO: Each rediscovery pair needs a sentence of evidence from both sides; the pairs in Fig. 5 are currently supported at the level of the catalog records only.]

D. Why explicit MPI became dominant, and why high-level partitioning has returned

We hypothesise that MPI won in Eras 3–4 because its cost model is transparent to the programmer and its performance does not depend on compiler maturity, whereas high-level partitioning has returned in Era 6 because ML programs are expressed as operator graphs with few, large, regular operators, which makes both propagation and cost modelling tractable. The HPF retrospective is the main evidence for the first half [75], and the propagation designs for the second [6], [7]. [TODO: Test both halves against the evaluation sections of [6], [7], [40] and the 1990s records.]

XIII. Open Problems

Each problem below is stated with the evidence in the catalog that motivates it.

- **Unified ownership and layout IR.** Affine distributions, mapping arrays, logical regions and mesh shardings are expressed in different IRs, and the MLIR shard dialect is the nearest candidate for a shared one [33], [88].
- **Dynamic and irregular partitioning.** Partitioners driven by connectivity or load and reuse of inspector results are shown in CHAOS and Kali, but reuse is guarded by conservative tests [28], [83].
- **Hierarchical heterogeneous systems.** Node, device and core levels are handled by nested mappings in Sequoia

and by inter-/intra-operator plans in Alpa, not by a general mechanism [40], [61].

- **Topology and memory capacity.** The analytic models in the catalog use communication volume without latency or bandwidth terms, and capacity appears only as a constraint [87].
- **Correctness and consistency.** Verification of generated communication is rarely addressed; the PartIR core is the exception, with a proof of its lowering reported by its authors [7].
- **Overlap.** Halo exchanges are sometimes non-blocking, but all-reduces are often blocking [9], [68].
- **Composability and debuggability.** The systems in the catalog compose through schedules or sharding annotations, but the catalog records no study of debugging distributed programs generated by them.
- **Wafer-scale and fine-grained local memory.** Placement with per-element memory and neighbour routing reopens the placement problem, and the wafer-scale stencil compilers handle only star-shaped stencils [9], [69].

[TODO: Each item needs at least one card-backed limitation statement; the first and second rest on records that are merged, the others are pending the cards of Legion, SpDISTAL and the wafer-scale papers.]

XIV. Conclusion

Distributed-memory compilation has been revisited for each new class of machine with private memories. The early systems of the late 1980s and 1990s made ownership a compiler-visible object, and HPF standardised a declarative distribution language that did not survive as a user-facing standard. Explicit MPI and PGAS languages carried the field through the 2000s. The current wave expresses distribution as sharding over meshes, with partitioning propagated by the compiler. The catalog, taxonomy and figures are maintained openly so that each new wave can begin from the records of the last rather than from scratch. [TODO: Finalise once the remaining cards are merged and the cross-era hypotheses are tested.]

References

[1] D. Callahan and K. Kennedy, "Compiling programs for distributed-memory multiprocessors," *The Journal of Supercomputing*, 1988.

[2] H. P. Zima, H.-J. Bast, and M. Gerndt, "Superb: A tool for semi-automatic mimd/simd parallelization," *Parallel Computing*, 1988.

[3] M. Chen, Y.-I. Choo, and J. Li, "Compiling parallel programs by optimizing performance," *The Journal of Supercomputing*, 1988.

[4] D. Loveman, "High performance fortran," *IEEE Parallel & Distributed Technology: Systems & Applications*, 1993.

[5] W. Gropp, E. Lusk, N. Doss, and A. Skjellum, "A high-performance, portable implementation of the mpi message passing interface standard," *Parallel Computing*, 1996.

[6] Y. Xu, H. Lee, D. Chen, B. Hechtman, Y. Huang, R. Joshi, M. Krikun, D. Lepikhin, A. Ly, M. Maggioni, R. Pang, N. Shazeer, S. Wang, T. Wang, Y. Wu, and Z. Chen, "Gspmd: General and scalable parallelization for ml computation graphs," arXiv preprint, 2021. [Online]. Available: https://arxiv.org/pdf/2105.04663

[7] S. Alabed, D. Belov, B. Chrzaszcz, J. Franco, D. Grewe, D. Maclaurin, J. Molloy, T. Natan, T. Norman, X. Pan, A. Paszke, N. A. Rink, M. Schaarschmidt, T. Sitdikov, A. Swietlik, D. Vytiniotis, and J. Wee, "Partir: Composing spmd partitioning strategies for machine learning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025.

[8] R. Yadav, A. Aiken, and F. Kjolstad, "Distal: the distributed tensor algebra compiler," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022.

[9] K. Rocki, D. V. Essendelft, I. Sharapov, R. Schreiber, M. Morrison, V. Kibardin, A. Portnoy, J. F. Dietiker, M. Syamlal, and M. James, "Fast stencil-code computation on a wafer-scale processor," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[10] H. Zima and B. Chapman, "Compiling for distributed-memory systems," *Proceedings of the IEEE*, 1993.

[11] S. Benkner and H. Zima, "Compiling high performance fortran for distributed-memory architectures," *Parallel Computing*, 1999.

[12] D. Unat, A. Dubey, T. Hoefler, J. Shalf, M. Abraham, M. Bianco, B. L. Chamberlain, R. Cledat, H. C. Edwards, H. Finkel, K. Fuerlinger, F. Hannig, E. Jeannot, A. Kamil, J. Keasler, P. H. J. Kelly, V. Leung, H. Ltaief, N. Maruyama, C. J. Newburn, and M. Pericas, "Trends in data locality abstractions for hpc systems," *IEEE Transactions on Parallel and Distributed Systems*, 2017.

[13] P. Lee, "Techniques for compiling programs on distributed memory multicomputers," *Parallel Computing*, 1995.

[14] J. W. Backus, H. Stern, I. Ziller, R. A. Hughes, R. Nutt, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, and P. B. Sheridan, "The fortran automatic coding system," *Papers presented at the February 26-28, 1957, western joint computer conference: Techniques for reliability on - IRE-AIEE-ACM '57 (Western)*, 1957.

[15] L. Lamport, "The parallel execution of do loops," *Communications of the ACM*, 1974.

[16] D. J. Kuck, R. H. Kuhn, D. A. Padua, B. Leasure, and M. Wolfe, "Dependence graphs and compiler optimizations," in *Proceedings of the 8th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '81*, 1981.

[17] R. Allen and K. Kennedy, "Automatic translation of fortran programs to vector form," *ACM Transactions on Programming Languages and Systems*, 1987.

[18] G. Barnes, R. Brown, M. Kato, D. Kuck, D. Slotnick, and R. Stokes, "The illiac iv computer," *IEEE Transactions on Computers*, 1968.

[19] R. M. Russell, "The cray-1 computer system," *Communications of the ACM*, 1978.

[20] J. B. Dennis and D. P. Misunas, "A preliminary architecture for a basic data-flow processor," *Proceedings of the 2nd annual symposium on Computer architecture - ISCA '75*, 1975.

[21] D. P. Misunas, "Report on the workshop on data flow computer and program organization," MIT Laboratory for Computer Science, Technical Memo MIT/LCS/TM-92, Tech. Rep., 1977, supplied copy; no public record located.

[22] Kung, "Why systolic architectures?" *Computer*, 1982.

[23] C. L. Seitz, "The cosmic cube," *Communications of the ACM*, 1985.

[24] W. D. Hillis and G. L. Steele, "Data parallel algorithms," *Communications of the ACM*, 1986.

[25] H. P. F. Forum, "High performance fortran language specification," *ACM SIGPLAN Fortran Forum*, 1993.

[26] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken, "Legion: Expressing locality and independence with logical regions," in *2012 International Conference for High Performance Computing, Networking, Storage and Analysis*, 2012.

[27] B. Chapman, P. Mehrotra, and H. Zima, "Vienna fortran: A fortran language extension for distributed memory multiprocessors," NASA Contractor Report 187634 / ICASE Report 91-72, Tech. Rep., 1991. [Online]. Available: https://ntrs.nasa.gov/citations/19920002452

[28] P. Mehrotra and J. V. Rosendale, "Programming distributed memory architectures using kali," NASA Contractor Report 187453 / ICASE Report 90-69, Tech. Rep., 1990. [Online]. Available: https://ntrs.nasa.gov/citations/19910002904

[29] C. Koelbel, "Compile-time generation of regular communications patterns," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 1991.

[30] S. Chatterjee, J. R. Gilbert, F. J. E. Long, R. Schreiber, and S.-H. Teng, "Generating local addresses and communication sets for data-parallel programs," in *Proceedings of the fourth ACM SIGPLAN symposium on Principles and practice of parallel programming - PPOPP '93*, 1993.

[31] U. Bondhugula, "Compiling affine loop nests for distributed-memory parallel architectures," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2013.

[32] N. Shazeer, Y. Cheng, N. Parmar, D. Tran, A. Vaswani, P. Koanantakool, P. Hawkins, H. Lee, M. Hong, C. Young, R. Sepassi, and B. Hechtman, "Mesh-tensorflow: Deep learning for supercomputers," arXiv preprint (also NeurIPS 2018), 2018. [Online]. Available: https://arxiv.org/pdf/1811.02084

[33] L. community, "Mlir 'shard' dialect (official documentation; formerly 'mesh')," MLIR documentation (mlir.llvm.org), 2025. [Online]. Available: https://mlir.llvm.org/docs/Dialects/Shard/

[34] J. Saltz, K. Crowley, R. Michandaney, and H. Berryman, "Run-time scheduling and execution of loops on message passing machines," *Journal of Parallel and Distributed Computing*, 1990.

[35] S. Ramaswamy, B. Simons, and P. Banerjee, "Optimizations for efficient array redistribution on distributed memory multicomputers," *Journal of Parallel and Distributed Computing*, 1996.

[36] D. J. Palermo, E. W. H. IV, and P. Banerjee, "Dynamic data partitioning for distributed-memory multicomputers," *Journal of Parallel and Distributed Computing*, 1996.

[37] R. Das, R. Ponnusamy, J. Saltz, and D. Mavriplis, "Distributed memory compiler methods for irregular problems: data copy reuse and runtime partitioning," NASA Contractor Report 187635 / ICASE Report 91-73, Tech. Rep., 1991. [Online]. Available: https://ntrs.nasa.gov/citations/19920002451

[38] Y. Hwang, B. Moon, S. D. Sharma, R. Ponnusamy, R. Das, and J. H. Saltz, "Runtime and language support for compiling adaptive irregular programs on distributed-memory machines," *Software: Practice and Experience*, 1995.

[39] A. Rogers and K. Pingali, "Compiling for distributed memory architectures," *IEEE Transactions on Parallel and Distributed Systems*, 1994.

[40] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing, J. E. Gonzalez, and I. Stoica, "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), 2022. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin

[41] C. Koelbel and P. Mehrotra, "Compiling global name-space parallel loops for distributed execution," *IEEE Transactions on Parallel and Distributed Systems*, 1991.

[42] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Compiling fortran d for mimd distributed-memory machines," *Communications of the ACM*, 1992.

[43] J. Li and M. Chen, "The data alignment phase in compiling programs for distributed-memory machines," *Journal of Parallel and Distributed Computing*, 1991.

[44] P. Banerjee, J. Chandy, M. Gupta, E. Hodges, J. Holm, A. Lain, D. Palermo, S. Ramaswamy, and E. Su, "The paradigm compiler for distributed-memory multicomputers," *Computer*, 1995.

[45] R. W. Numrich and J. Reid, "Co-array fortran for parallel programming," *ACM SIGPLAN Fortran Forum*, 1998.

[46] K. Yelick, L. Semenzato, G. Pike, C. Miyamoto, B. Liblit, A. Krishnamurthy, P. Hilfinger, S. Graham, D. Gay, P. Colella, and A. Aiken, "Titanium: a high-performance java dialect," *Concurrency: Practice and Experience*, 1998.

[47] P. Tang and J. Xue, "Generating efficient tiled code for distributed memory machines," *Parallel Computing*, 2000.

[48] W.-Y. Chen, D. Bonachea, J. Duell, P. Husbands, C. Iancu, and K. Yelick, "A performance analysis of the berkeley upc compiler," in *Proceedings of the 17th annual international conference on Supercomputing*, 2003.

[49] M. Kruse, "Introducing molly: Distributed memory parallelization with llvm," arXiv preprint, 2014. [Online]. Available: https://arxiv.org/pdf/1409.2088

[50] U. Bondhugula, "Automatic distributed-memory parallelization and code generation using the polyhedral framework," IISc-CSA-TR-2011-3, Indian Institute of Science, Tech. Rep., 2011. [Online]. Available: https://www.csa.iisc.ac.in/~udayb/publications/uday11distmem-tr.pdf

[51] M. Gerndt, "Updating distributed variables in local computations," *Concurrency: Practice and Experience*, 1990.

[52] J. Saltz, H. Berryman, and J. Wu, "Multiprocessors and run-time compilation," *Concurrency: Practice and Experience*, 1991.

[53] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Compiler optimizations for fortran d on mimd distributed-memory machines," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 1991.

[54] B. Chapman, P. Mehrotra, and H. Zima, "Programming in vienna fortran," *Scientific Programming*, 1992.

[55] M. Gupta and P. Banerjee, "Demonstration of automatic data partitioning techniques for parallelizing compilers on multicomputers," *IEEE Transactions on Parallel and Distributed Systems*, 1992.

[56] E. Su, D. J. Palerrno, and P. Banerjee, "Automating parallelism of regular computations for distributed-memory multicomputers in the paradigm compiler," in *1993 International Conference on Parallel Processing - ICPP'93 Vol2*, 1993.

[57] Z. Bozkus, A. Choudhary, G. Fox, T. Haupt, and S. Ranka, "Fortran 90d/hpf compiler for distributed memory mimd computers," in *Proceedings of the 1993 ACM/IEEE conference on Supercomputing - Supercomputing '93*, 1993.

[58] J. Harris, J. A. Bircsak, M. R. Bolduc, J. A. Diewald, I. Gale, N. W. Johnson, S. Lee, C. A. Nelson, and C. D. Offner, "Compiling high performance fortran for distributed-memory systems," *Digital Technical Journal*, 1995. [Online]. Available: https://ftp.math.utah.edu/pub/tex/bib/idx/dectechj/7/3/5_23.html

[59] P. Charles, C. Grothoff, V. Saraswat, C. Donawa, A. Kielstra, K. Ebcioglu, C. von Praun, and V. Sarkar, "X10: an object-oriented approach to non-uniform cluster computing," *Proceedings of the 20th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, 2005.

[60] B. Chamberlain, D. Callahan, and H. Zima, "Parallel programmability and the chapel language," *The International Journal of High Performance Computing Applications*, 2007.

[61] K. Fatahalian, T. Knight, M. Houston, M. Erez, D. Horn, L. Leem, J. Park, M. Ren, A. Aiken, W. Dally, and P. Hanrahan, "Sequoia: Programming the memory hierarchy," in *ACM/IEEE SC 2006 Conference (SC'06)*, 2006.

[62] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan, "A practical automatic polyhedral parallelizer and locality optimizer," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2008.

[63] E. Slaughter, W. Lee, S. Treichler, M. Bauer, and A. Aiken, "Regent: a high-productivity programming language for hpc with logical regions," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015.

[64] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," Proceedings of Machine Learning and Systems (MLSys) 1, 2019. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/b422680f3db0986ddd7f8f126baaf0fa-Abstract.html

[65] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "Gshard: Scaling giant models with conditional computation and automatic sharding," arXiv preprint (also ICLR 2021), 2020. [Online]. Available: https://arxiv.org/pdf/2006.16668

[66] M. Jacquelin, M. Araya–Polo, and J. Meng, "Scalable distributed high-order stencil computations," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

[67] R. Yadav, A. Aiken, and F. Kjolstad, "Spdistal: Compiling distributed sparse tensor computations," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

[68] G. Bisbas, A. Lydike, E. Bauer, N. Brown, M. Fehr, L. Mitchell, G. Rodriguez-Canal, M. Jamieson, P. H. J. Kelly, M. Steuwer, and T. Grosser, "A shared compilation stack for distributed-memory parallelism in stencil dsls," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024.

[69] N. Stawinoga, D. Katz, A. Lydike, J. Zarins, N. Brown, G. Bisbas, and T. Grosser, "An mlir lowering pipeline for stencils at wafer-scale," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '26)*, 2026.

[70] O. Project, "Shardy: an mlir-based tensor partitioning system (official documentation)," OpenXLA project documentation, 2024. [Online]. Available: https://openxla.org/shardy

[71] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Evaluation of compiler optimizations for fortran d on mimd distributed memory machines," in *Proceedings of the 6th international conference on Supercomputing - ICS '92*, 1992.

[72] E. Su, A. Lain, S. Ramaswamy, D. J. Palermo, E. W. H. IV, and P. Banerjee, "Advanced compilation techniques in the paradigm compiler for distributed-memory multicomputers," in *Proceedings of the 9th International Conference on Supercomputing (ICS '95)*, 1995.

[73] M. Gupta, S. Midkiff, E. Schonberg, V. Seshadri, D. Shields, K.-Y. Wang, W.-M. Ching, and T. Ngo, "An hpf compiler for the ibm sp2," in *Proceedings of the 1995 ACM/IEEE conference on Supercomputing (CDROM) - Supercomputing '95*, 1995.

[74] M. P. I. Forum, "Mpi: A message-passing interface standard (version 1.0)," MPI Forum, 1994. [Online]. Available: https://www.mpi-forum.org/docs/

[75] K. Kennedy, C. Koelbel, and H. Zima, "The rise and fall of high performance fortran," in *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, 2007.

[76] K. Yelick, P. Hilfinger, S. Graham, D. Bonachea, and J. Su, "Parallel languages and compilers: Perspective from the titanium experience," *The International Journal of High Performance Computing Applications*, vol. 21, no. 3, 2007.

[77] M. M. Baskaran, U. Bondhugula, S. Krishnamoorthy, J. Ramanujam, A. Rountev, and P. Sadayappan, "Automatic data movement and computation mapping for multi-level parallel architectures with explicitly managed memories," in *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and practice of parallel programming*, 2008.

[78] R. Dathathri, "Compiling for a dataflow runtime on distributed-memory parallel architectures," Master's thesis, Indian Institute of Science, 2014, m.Sc. (Engineering) thesis.

[79] R. Dathathri, R. T. Mullapudi, and U. Bondhugula, "Compiling affine loop nests for a dynamic scheduling runtime on shared and distributed memory," *ACM Transactions on Parallel Computing*, 2016.

[80] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," arXiv preprint (also NeurIPS 2019), 2018. [Online]. Available: https://arxiv.org/pdf/1811.06965

[81] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, "Pipedream: generalized pipeline parallelism for dnn training," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.

[82] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," arXiv preprint, 2019. [Online]. Available: https://arxiv.org/pdf/1909.08053

[83] R. Ponnusamy, J. Saltz, A. Choudhary, Y.-S. Hwang, and G. Fox, "Runtime support and compilation methods for user-specified irregular data distributions," *IEEE Transactions on Parallel and Distributed Systems*, 1995.

[84] R. Chandra, D.-K. Chen, R. Cox, D. E. Maydan, N. Nedeljkovic, and J. M. Anderson, "Data distribution support on distributed shared memory multiprocessors," in *Proceedings of the ACM SIGPLAN 1997 Conference on Programming Language Design and Implementation (PLDI '97)*, 1997.

[85] Y. N. Shen, R. C. Zhao, and L. Wang, "Eliminating redundant communication of code generation for distributed memory machines," in *Eighth ACIS International Conference on Software Engineering, Artificial Intelligence, Networking, and Parallel/Distributed Computing (SNPD 2007)*, 2007.

[86] S. P. Amarasinghe and M. S. Lam, "Communication optimization and code generation for distributed memory machines," in *Proceedings of the ACM SIGPLAN 1993 conference on Programming language design and implementation*, 1993.

[87] M. Kong, R. A. Yosef, A. Rountev, and P. Sadayappan, "Automatic generation of distributed-memory mappings for tensor computations," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023.

[88] M. Vaidya, S. Singh, D. Mantri, M. S. Eydenberg, B. M. Kelley, S. Rajamanickam, A. Rountev, and P. Sadayappan, "Distributed sparse tensor computations in mlir," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025.

[89] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[90] R. Baghdadi, J. Ray, M. B. Romdhane, E. D. Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe, "Tiramisu: A polyhedral compiler for expressing fast and portable code," in *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2019.

TABLE II
CORE SYSTEMS IN THE CATALOG. A QUESTION MARK MARKS A CELL NOT YET VERIFIED FROM A PRIMARY SOURCE.

System	Refs	Years	Input	Distribution	Work mapping	Communication	Runtime	Targets	Artifa
SUPERB	[2], [51]	1986-1992	Fortran 77 + interactive distribution	?	?	?	SUPRENUM MP	SUPRENUM	unavai
Kali	[28], [41]	1989-1992	Fortran + dist + doall/on	block, cyclic, block_cyclic, user map, dynamic	on clause / owner()	in/out sets static or run-time inspector	cached inspector/executor	iPSC-class multicomputers	unavai
PARTI / CHAOS	[34], [52]	1989-1997	irregular loops with index arrays	mapping arrays via translation tables; partitioner-driven	iteration partitioning via RIG/partitioner	inspector-built import/export schedules; schedule reuse	PARTI/CHAOS	iPSC/2, iPSC/860, CM-2	unavai
Fortran D	[1], [53]	1991-1996	Fortran 77 + decomposition directives	?	?	?	NX / PARTI	iPSC/860	unavai
Vienna Fortran / VFCS	[27], [54]	1991-1999	Fortran 77 + DIST/ALIGN	block, cyclic(k), user-defined, INDIRECT, dynamic	owner computes; FORALL ON	overlap areas, vectorised messages (VFCS)	VFCS	iPSC/860, GENESIS, SUPRENUM	unavai
PARADIGM	[55], [56]	1992-1997	Fortran 77 (automatic)	?	?	?	?	iPSC, Paragon, CM-5 (to verify)	unavai
Fortran 90D/HPF (Syracuse)	[57]	1993-1995	Fortran 90D	?	?	runtime collective library	F90D runtime	?	unavai
High Performance Fortran (language)	[4], [25]	1993-2000	Fortran 90 + directives	BLOCK, CYCLIC, CYCLIC(k), replicated; 2.0 adds INDIRECT, GEN_BLOCK	implementation-defined (owner computes typical)	implementation-defined	n/a	n/a	specifi
Digital (DEC) Fortran 90 HPF compiler	[58]	1994-1998	HPF 1.1	BLOCK, CYCLIC, replicated, alignment	data-parallel; owner computes not strictly enforced	region assignments; vectorised sends; shadow edges; inlined reductions	PSE SEND/RECEIVE	Alpha workstation farms, AlphaServer SMP	comm unavai
Titanium	[46]	1998-2010	Titanium (Java dialect)	?	SPMD	?	GASNet	clusters	archiv
Co-array Fortran	[45]	1998-present	Fortran + [ ] codimensions	one co-array per image	SPMD, programmer	explicit co-array references	?	Cray T3E and later	spec; 2008
Unified Parallel C (Berkeley UPC)	[48]	2002-present	UPC	?	upc_forall affinity	?	GASNet	clusters	open s
X10	[59]	2004-2020	X10	?	at(place) / async	?	X10RT	clusters	archiv
Chapel	[60]	2005-present	Chapel	user-defined domain maps (Block, Cyclic, BlockCyclic, Replicated, ...)	forall over distributed domains; on clauses	compiler + runtime (GET/PUT, aggregation)	Chapel runtime	clusters, GPUs	open s
Sequoia	[61]	2006-2010	task trees	?	task-to-memory-level mapping	bulk copies between levels	Sequoia runtime	Cell, clusters	unavai
Pluto distributed-memory code generation (Bondhugula et al.)	[50], [62]	2011-2016	C affine loop nests	derived from computation partition (flow-out sets)	polyhedral iteration-space partitioning	polyhedral communication sets to MPI	MPI / dynamic scheduler	InfiniBand clusters	open (pluto distme branch
Legion / Regent	[26], [63]	2012-present	tasks + logical regions	region partitions chosen by program/mapper	mapper interface	runtime-managed coherence and copies	Legion/Realm	GPU clusters	open s
Molly	[49]	2014-2015	C++ + layout declarations	programmer-declared	polyhedral	generated asynchronous MPI	MPI	Blue Gene/Q (intended)	archiv
Mesh-TensorFlow	[32]	2018-2021	TF graph + named dims + layout	dimension-to-mesh-axis layout	SPMD per layout	allreduce/allgather/alltoall derived from layouts	TF/TPU	TPUv2 pods	archiv
FlexFlow	[64]	2018-present	operator graph	SOAP output-tensor partitions	MCMC search with simulator	task graph with transfer tasks	FlexFlow/Legion	P100/K80 clusters	open s
XLA SPMD partitioner (GShard / GSPMD)	[6], [65]	2020-present	HLO + sharding annotations	replicated / tiled / partially tiled per tensor over device mesh	SPMD; sharding propagation	collectives + halo exchange + resharding inserted per operator	XLA	TPUv3 pods (GPU supported)	open (XLA)
Cerebras WSE programming (CSL / hand-mapped stencils)	[9], [66]	2020-present	CSL	programmer-placed per-PE tiles	spatial placement	fabric colours / routes configured by programmer	SDK	CS-1/CS-2/CS-3	comm SDK
PartIR	[7]	2021-2025	StableHLO + tactics	?	?	?	XLA	TPU, GPU	?
DISTAL / SpDISTAL	[8], [67]	2022-2023	tensor index notation + formats + schedule	tensor distribution notation over a processor grid; sparse non-zero/universe partitions	distribute/communicate/rotate schedule to index-task launches	Legion partitions (bounds analysis; image/preimage for sparse)	Legion	Lassen CPU/GPU nodes	open s
Alpa	[40]	2022-2023	JAX graph	per-stage sharding specs (ILP)	DP over stages/submeshes + ILP intra-op	GSPMD collectives + cross-mesh resharding	Alpa MPMD runtime	64x V100	archiv
MLIR shard dialect (formerly mesh)	[33]	2023-present	MLIR + shard.grid/shard.sharding	per-axis grid sharding with halos	SPMD	all_gather, all_reduce, all_to_all, reduce_scatter, update_halo, send/recv	n/a	n/a	upstre MLIR
xDSL stencil / dmp / mpi stack	[68], [69]	2023-present	stencil DSLs	static Cartesian rank-grid decomposition (dmp)	owner computes per block; one z-column per PE on WSE	dmp.swap halo exchange to mpi dialect or CSL chunked prefetch	MPI / CSL library	ARCHER2, GPUs, FPGA, CS-2/CS-3	open s

TABLE III
DECOMPOSITION SUPPORT BY SYSTEM. ● SUPPORTED, ◐ PARTIAL, ○ NOT SUPPORTED, ? NOT YET VERIFIED.

System	block	cyclic	block-cyclic	replication	halo
SUPERB	?	?	?	?	?
Kali	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$
PARTI / CHAOS	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$circ$	\{}$circ$
Fortran D	?	?	?	?	?
Vienna Fortran / VFCS	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$
PARADIGM	?	?	?	?	?
Fortran 90D/HPF (Syracuse)	?	?	?	?	?
High Performance Fortran (language)	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$circ$
Digital (DEC) Fortran 90 HPF compiler	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$
Titanium	?	?	?	?	?
Co-array Fortran	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
Unified Parallel C (Berkeley UPC)	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$circ$	\{}$circ$
X10	?	?	?	?	?
Chapel	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$circ$
Sequoia	?	?	?	?	?
Pluto distributed-memory code generation (Bondhugula et al.)	\{}$bullet$	\{}$circ$\{}!\{}!\{}$bullet$	\{}$circ$\{}!\{}!\{}$bullet$	\{}$bullet$	\{}$bullet$
Legion / Regent	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$circ$	\{}$circ$
Molly	\{}$bullet$	?	?	?	?
Mesh-TensorFlow	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
FlexFlow	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
XLA SPMD partitioner (GShard / GSPMD)	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
Cerebras WSE programming (CSL / hand-mapped stencils)	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$circ$	\{}$bullet$
PartIR	?	?	?	?	?
DISTAL / SpDISTAL	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
Alpa	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
MLIR shard dialect (formerly mesh)	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$bullet$
xDSL stencil / dmp / mpi stack	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$circ$	\{}$circ$
Shardy (OpenXLA)	\{}$bullet$	?	?	\{}$bullet$	?

TABLE IV
COMPILER ARCHITECTURE BY SYSTEM (FROM CATALOG TAGS).

System	src-to-src	polyhedral	grap
SUPERB	\{}$bullet$	\{}$circ$	\{}
Kali	\{}$bullet$	\{}$circ$	\{}
PARTI / CHAOS	\{}$circ$	\{}$circ$	\{}
Fortran D	\{}$bullet$	\{}$circ$	\{}
Vienna Fortran / VFCS	\{}$bullet$	\{}$circ$	\{}
PARADIGM	\{}$bullet$	\{}$circ$	\{}
Fortran 90D/HPF (Syracuse)	\{}$bullet$	\{}$circ$	\{}
High Performance Fortran (language)	\{}$circ$	\{}$circ$	\{}
Digital (DEC) Fortran 90 HPF compiler	\{}$circ$	\{}$circ$	\{}
Titanium	\{}$bullet$	\{}$circ$	\{}
Co-array Fortran	\{}$circ$	\{}$circ$	\{}
Unified Parallel C (Berkeley UPC)	\{}$bullet$	\{}$circ$	\{}
X10	\{}$bullet$	\{}$circ$	\{}
Chapel	\{}$bullet$	\{}$circ$	\{}
Sequoia	\{}$circ$	\{}$circ$	\{}
Pluto distributed-memory code generation (Bondhugula et al.)	\{}$bullet$	\{}$bullet$	\{}
Legion / Regent	\{}$bullet$	\{}$circ$	\{}
Molly	\{}$circ$	\{}$bullet$	\{}
Mesh-TensorFlow	\{}$circ$	\{}$bullet$	\{}b
FlexFlow	\{}$circ$	\{}$circ$	\{}b
XLA SPMD partitioner (GShard / GSPMD)	\{}$circ$	\{}$circ$	\{}b
Cerebras WSE programming (CSL / hand-mapped stencils)	\{}$circ$	\{}$circ$	\{}
PartIR	\{}$circ$	\{}$circ$	\{}
DISTAL / SpDISTAL	\{}$circ$	\{}$circ$	\{}
Alpa	\{}$circ$	\{}$circ$	\{}b
MLIR shard dialect (formerly mesh)	\{}$circ$	\{}$circ$	\{}
xDSL stencil / dmp / mpi stack	\{}$circ$	\{}$circ$	\{}
Shardy (OpenXLA)	\{}$circ$	\{}$circ$	\{}

TABLE V
EVALUATION METHODOLOGY BY SYSTEM. A QUESTION MARK MARKS A CELL NOT YET EXTRACTED FROM THE PRIMARY SOURCE.

System	Benchmarks	Baseline	Scale	Metrics	Productivity eval.	Artifact
SUPERB	?	?	?	?	?	unavailable
Kali	?	?	?	?	?	unavailable
PARTI / CHAOS	?	?	?	?	?	unavailable
Fortran D	?	?	?	?	?	unavailable
Vienna Fortran / VFCS	?	?	?	?	?	unavailable
PARADIGM	?	?	?	?	?	unavailable
Fortran 90D/HPF (Syracuse)	?	?	?	?	?	unavailable
High Performance Fortran (language)	?	?	?	?	?	specification
Digital (DEC) Fortran 90 HPF compiler	?	?	?	?	?	commercial, unava
Titanium	?	?	?	?	?	archived
Co-array Fortran	?	?	?	?	?	spec; Fortran 2008 rays
Unified Parallel C (Berkeley UPC)	?	?	?	?	?	open source
X10	?	?	?	?	?	archived
Chapel	?	?	?	?	?	open source
Sequoia	?	?	?	?	?	unavailable
Pluto distributed-memory code generation (Bondhugula et al.)	?	?	?	?	?	open source (pluto mem branch)
Legion / Regent	?	?	?	?	?	open source
Molly	?	?	?	?	?	archived
Mesh-TensorFlow	?	?	?	?	?	archived
FlexFlow	?	?	?	?	?	open source
XLA SPMD partitioner (GShard / GSPMD)	?	?	?	?	?	open source (XLA
Cerebras WSE programming (CSL / hand-mapped stencils)	?	?	?	?	?	commercial SDK
PartIR	?	?	?	?	?	?
DISTAL / SpDISTAL	?	?	?	?	?	open source
Alpa	?	?	?	?	?	archived
MLIR shard dialect (formerly mesh)	?	?	?	?	?	upstream MLIR
xDSL stencil / dmp / mpi stack	?	?	?	?	?	open source
Shardy (OpenXLA)	?	?	?	?	?	open source